T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	31/07/2026	**DATE SUBMITTED:**	05/08/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 4.1 **[Stacks]**

1. PROCEDURE OUTPUT

[Source Code]

```
#include <iostream>
#include "stackList.h"

int main() {
    std::cout << " >>> Pushing 'C' 'P' 'E' onto the stack <<< \n";

    push('C');
    push('P');
    push('E');

    Top<char>();
    displayAll<char>();
    std::cout << "\n >>> Popping all the elements <<< \n";

    while(!isEmpty<char>()){
        std::cout <<" Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n>>> Testing the Underflow <<< \n";
    pop<char>();

    return 0;
}
```

[Header File]

```
#ifndef STACK_LIST_H
#define STACK_LIST_H

template <typename T>
class Node {
    public:
    T data;
    Node* next;
};

template <typename T>
Node<T> *head = nullptr;


template <typename T>
void push(T newData){

    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    newNode->next = head<T>;
    head<T> = newNode;
};
```

```cpp
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}


template <typename T>
T pop(){

    if(head<T> == nullptr){
        std::cout << "Stack overflow" << std::endl;
    }

    Node<T>* temp = head<T>;
    T tempVal = temp->data;

    head<T> = head<T>->next;

    delete temp;
    return tempVal;
}


template <typename T>
void Top(){

    if(isEmpty<T>()){
        std::cout << "[ Stack underflow ]" <<std::endl;
        return;
    }

    std::cout <<"Top of the Stack: " << head<T>->data << std::endl;
}
```

```
61  template <typename T>
62  void displayAll(){
63      if(head<T> == nullptr) {
64          std::cout << "Stack underflow\n";
65          return;
66      }
67      std::cout << "Displaying the stack: \n";
68      Node<T>* temp = head<T>;
69      while(temp != nullptr){
70          std::cout << temp->data << "\n";
71          temp = temp->next;
72      }
73      std::cout <<"\n";
74  }
75
76  #endif
```

[Output]

```
 >>> Pushing 'C' 'P' 'E' onto the stack <<<
Top of the Stack: E
Displaying the stack:
E
P
C


 >>> Popping all the elements <<<
 Popped: E
 Popped: P
 Popped: C

>>> Testing the Underflow <<<
Stack overflow
```

[Analysis]

In the procedural activity, we were tasked to include a function that will display all the elements in the stack. As the code was provided to us, it is for us to further understand the logic behind the stacks of each operation in terms of linked list. As the code represented above, In the main cpp file, includes the function callers and the elements that were pushed inside the stacks. The **top() function** implies what is the top element of the stack. While the **DisplayAll() function** initializes that in the **header file** method, you equate **temp** into the **head<T>** where it will serve as the reader where if it is not nullptr, it will print the current data element inside the temp and proceed to the

next point until it reaches the **nullptr**. Thus, explaining the main functionality of the DisplayAll() function which is the main task of this activity, which is to understand the flow procedure of the code provided and give an analysis to how the entire code works.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Stack Using Array

[CPP File Code]

```cpp
#include <iostream>
#include <string>
#include "ArrayStack.h"

bool checkArrBalance(const std::string& expr) {
    ArrayStack<char> stack(expr.length());

    for (char ch : expr) {
        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (stack.isEmpty()) return false;

            char open = stack.pop();
            if ((ch == ')' && open != '(') ||
                (ch == '}' && open != '{') ||
                (ch == ']' && open != '[')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

```cpp
int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B)+[C-D]}"
    };

    std::cout << "=== TEMPLATE ARRAY STACK ===\n\n";
    for (const auto& expr : expressions) {
        std::cout << "Expression: " << expr << " -> "
                  << (checkArrBalance(expr) ? "Valid" : "Invalid")
    }
    return 0;
}
```

[Header File]

```cpp
#ifndef ARRAY_STACK_H
#define ARRAY_STACK_H
#include <iostream>

template <typename T>
class ArrayStack {
private:
    int topIndex;
    int capacity;
    T* arr;

public:
    ArrayStack(int size) {
        capacity = size;
        arr = new T[capacity];
        topIndex = -1;
    }

    ~ArrayStack() {
        delete[] arr;
    }

    bool isEmpty() {
        return topIndex == -1;
    }

    void push(T val) {
        if (topIndex < capacity - 1) {
            arr[++topIndex] = val;
        }
    }
```

```
32
33      T pop() {
34          if (!isEmpty()) {
35              return arr[topIndex--];
36          }
37          return T();
38      }
39  };
40
41  #endif
```

[Output]

```
======== ARRAY STACK ========

Expression: (A+B)+(C-D) -> Valid
Expression: ((A+B)+(C-D) -> Invalid
Expression: ((A+B)+[C-D]) -> Valid
Expression: ((A+B)+[C-D]} -> Invalid
PS C:\Users\Striker\Documents\CPE21S4\CPE010>
```

[Analysis]

In the ILO C, we are tasked to create 2 implementations using stacks to check the balancing of symbols. Which was **Stack using Arrays**, and **Stack using Linked Lists**. In this analysis, we will be tackling the Stack using Array implementation for checking the balancing of symbols. Let's first proceed to the cpp file. In the first line of codes 5 - 23, is the entire bool function that processes the condition of checking the array's balancing. It is named as **checkArrBalance(),** inside the function includes the initialization of the **array stack size indicated** as **ArrayStack<char> stack(expr.length())**. After the initialization of the array stack size, it consists of a for loop function for checking each character inside the expression. Inside the for loop function, consisting of the if else statements, the if indicates that **IF ch** is an **opening bracket**, it **pushes it**. If **NO** it **skips it**. And after the first verification, there's another **else-if function** containing the same condition but in the closing brackets, where **IF** it is a **closing brackets**, It will **pop** and **verify the character**. If NO, as the same else function is initialized, it skips it. And inside the else-if function, actually represents a **structure if-else function** since it consists of another if condition where if stack is empty indicated as **if(stack.isEmptyt*())**, it will return **false / error**. The next if function is where it indicates if the **popped symbol does not match**, it should **report an error**. And lastly, the return **stack.isEmpty()** where it returns true if the stack is truly empty.

In the main function, line code 25 to 39 is the **string initialization** of the **expression array**. Where lays all the

needed conditions to be checked if it has a balanced expression and if it is a valid one.

Meanwhile in the header file, consist of the methods and deconstructor of arraystack. The header file handles the function that happens in the identification on the bool if it is empty, indicated as **bool isEmpty()**. The push and pop function, has their own condition, where in the push function, consist an if structure where a parameter of val increments the topIndex if the condition meets that **topIndex** is **less than capacity - 1.** While the pop function consists of a condition where IF it is **not isEmpty()** it will decrement the **topindex.**

Stack Using Linked List

[CPP File Code]

```cpp
#include <iostream>
#include <string>
#include "ListStack.h"

bool checkBalanceList(const std::string& expr) {
    ListStack<char> stack;

    for (char ch : expr) {
        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (stack.isEmpty()) return false;

            char open = stack.pop();
            if ((ch == ')' && open != '(') ||
                (ch == '}' && open != '{') ||
                (ch == ']' && open != '[')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

```cpp

int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B)+[C-D]}"
    };

    std::cout << "======== LINKED LIST STACK ========\n\n";
    for (const auto& expr : expressions) {
        std::cout << "Expression: " << expr << " -> "
                  << (checkBalanceList(expr) ? "Valid" : "Invalid") << "\n";
    }
    return 0;
}
```

[Header File]

```
#ifndef LIST_STACK_H
#define LIST_STACK_H
#include <iostream>

template <typename T>
class ListStack {
private:
    struct Node {
        T data;
        Node* next;
    };
    Node* topNode = nullptr;

public:
    ~ListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    bool isEmpty() {
        return topNode == nullptr;
    }

    void push(T val) {
        topNode = new Node{val, topNode};
    }
```

```
28
29      T pop() {
30          if (isEmpty()) return T();
31          Node* temp = topNode;
32          T val = temp->data;
33          topNode = topNode->next;
34          delete temp;
35          return val;
36      }
37  };
38
39  #endif
```

[Output]

```
======== LINKED LIST STACK ========

Expression: (A+B)+(C-D) -> Valid
Expression: ((A+B)+(C-D) -> Invalid
Expression: ((A+B)+[C-D]) -> Valid
Expression: ((A+B)+[C-D]} -> Invalid
```

[Analysis]

Similarly in the array stack, I decided to use the same format of balancing and checking of the expression. The differences are the process of the method happening inside the header file. The only new in the CPP file is the **ListStack<char> stack;** which directly creates an empty stack. As for the recap, The indication of the functions in the CPP file, is a loop approach to check and balance through the expressions.

In the header file of the Linked List stack implementation, we first **created an object to initialize** and implement through the **processing of scanning between nodes** of the linked list of the stack created. Declaring variables and addresses next to next to scroll through and verify the entire expression.

In the function of **listStack()** a condition was made that if the stack is **NOT empty**, therefore, it will **pop** the specific expression. In the second function implemented in the public section, where a condition of true or false, if the expression or node to be specific in terms of linked list, has no value, it initializes nullptr.

In the **pop() function**, a conditional function is initialized that if the stack is empty, it will return an error. Otherwise, it processes the **popping of elements**, where a **temp node** is created, initialized to **topNode**, and its value is initialized to the newly created **T val object.** WHILE, the **temp** became the new **topNode**, the **previous topNode transfers** to the **next node**. After the **temp node pops**, it **deletes** the **temp node**, and it **returns** the **value of the temp node**. That's entirely the difference between the 2 stack implementation in terms of array and linked list in the creation of my supplementary activity.

Expression	Valid? (Y/N)	Output	Analysis
(A+B)+(C-D)	Y	Expression: (A+B)+(C-D) -> Valid	The expression shows a balanced symbol and character, it both has no lacking open and close symbols.
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) -> Invalid	The expression shows an imbalance in the opening symbol, in which there is an extra open parenthesis without a close parenthesis.
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) -> Valid	Expression shows valid and balanced symbols and character, it both has their own respective open and close symbols and no lacking symbols are existing.
((A+B]+[C-D]}	N	Expression: ((A+B)+[C-D]} -> Invalid	Expression shows an invalid and imbalance expression as the expression contains a closing bracket, without a pair of opening brackets.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I used AI to further explain to me how ILO C is structured, how it is implemented through both stack of arrays and linked list to be able to integrate the learnings I have adapted and the concept that I have learned towards the AI.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.